

Write up ini, dan solvenya, AI generated

WRITEUP CTF - CRYPTO: "MATH" (C2SI)

FLAG

C2SI{e4sy_m4th_w3lc0m3_t0_ctf_f0r_n3w_m3mber_4t_SMK_Immanuel_Pontianak_G00DLUCK}

1. ANALISIS SOAL

```
1 from fractions import Fraction
2 import numpy as np
3 import json
4
5 with open("flag.txt","r") as f:
6     flag = f.read().strip()
7
8 chunk_size = len(flag) // 4
9 chunks = [flag[i * chunk_size : (i + 1) * chunk_size] for i in range(4)]
10
11 part1 = [ord(c) for c in chunks[0]]
12 part2 = [ord(c) for c in chunks[1]]
13 part3 = [ord(c) for c in chunks[2]]
14 part4 = [ord(c) for c in chunks[3]]
15
16 # Definite Integral
17 def calculus(data_ord):
18     n = len(data_ord)
19     cipher_integrals = []
20
21     for k in range(1, n + 1):
22         val = sum(Fraction(a, i + 1) * (k ** (i + 1)) for i, a in enumerate(data_ord))
23         cipher_integrals.append(float(val))
24
25     return cipher_integrals
26
27 key_matrix_4x4 = np.array([
28     [3, 2, 1, 5],
29     [4, 6, 4, 3],
30     [6, 7, 6, 4],
31     [2, 3, 4, 6]
32 ], dtype=np.int64)
33
34 def matrix(data_ord):
35     padded_data = data_ord
36
37     P = np.array(padded_data, dtype=np.int64).reshape((4,5))
38
39     C = np.dot(key_matrix_4x4, P)
40
41     return C.tolist(), key_matrix_4x4.tolist()
42
43 def algebra(data_ord):
44     n = len(data_ord)
45     np.random.seed(1337)
46
47     A = np.random.randint(1, 10, size=(n, n))
48
49     while int(round(np.linalg.det(A))) == 0:
50         A = np.random.randint(1, 10, size=(n,n))
51
52     b = np.random.randint(10, 50, size=n)
53
54     x = np.array(data_ord, dtype=np.int64)
55
56     y = np.dot(A,x) + b
57
58     return {
59         "A": A.tolist(),
60         "b": b.tolist(),
61         "y": y.tolist()
62     }
63
64 def statistics(data_ord):
65     n = len(data_ord)
66     cumulative_means = []
67     running_sum = 0
68
69     for k in range(1, n + 1):
70         running_sum += data_ord[k - 1]
71         cumulative_means.append(running_sum / k)
72
73     return {
74         "cumulative_mean": cumulative_means
75     }
76
77 output_data = {
78     "part1_calculus": calculus(part1),
79     "part2_matrix": matrix(part2),
80     "part3_linear_algebra": algebra(part3),
81     "part4_statistics": statistics(part4)
82 }
83
84 with open("enc_flag.txt","w") as f:
85     json.dump(output_data, f, indent=4)
86
87 print("done!")
```

Screenshot: isi lengkap file enc.py

File enc.py memotong flag (80 karakter) menjadi 4 chunk masing-masing 20 karakter, lalu tiap chunk di-encode dengan operasi matematika yang berbeda. Semua operasinya linear dan invertible: tidak ada key rahasia dan tidak ada operasi satu arah. Jadi tiap part cukup dibalik.

Ringkasan keempat part:

Part 1 - Integral tentu polinomial - dibalik dengan sistem linear tipe Vandermonde, diselesaikan lewat DFS + interval pruning

Part 2 - Perkalian matriks $C = K \times P$, dengan K diberikan - dibalik dengan $P = K^{-1} \times C$

Part 3 - $y = Ax + b$, dengan A dan b diberikan - dibalik dengan solve $Ax = y - b$

Part 4 - Rata-rata kumulatif - dibalik dengan selisih dari sum kumulatif

Catatan penting: matriks A dan vektor b pada Part 3 dibangkitkan dengan `np.random.seed(1337)` yang nilainya fixed, dan keduanya ikut ditulis ke file output. Artinya Part 3 sama sekali tidak menyimpan rahasia.

2. PART 4 - CUMULATIVE MEAN

Rumus enkripsinya sederhana:

$$\text{mean}[k] = (a_0 + a_1 + \dots + a_k) / (k + 1)$$

Kalikan balik untuk mendapat sum kumulatif, lalu ambil selisih antar suku berurutan.

```

1  # ----- Part 4: rata-rata kumulatif -----
2  def solve_part4():
3      cm = DATA["part4_statistics"]["cumulative_mean"]
4      out, prev = [], 0
5      for k, m in enumerate(cm, 1):
6          s = round(m * k)
7          out.append(s - prev)
8          prev = s
9      return "".join(map(chr, out))
10
11
12  if __name__ == "__main__":
13      parts = [solve_part1(), solve_part2(), solve_part3(), solve_part4()]
14      for i, p in enumerate(parts, 1):
15          print(f"part{i}: {p}")
16      print("\nFLAG:", "".join(parts))
17

```

Screenshot: fungsi `solve_part4()` di `solve.py`

Hasil: `_Pontianak_G00DLUCK}`

3. PART 3 - LINEAR ALGEBRA

Persamaannya $y = Ax + b$, dengan matriks A berukuran 20×20 dan vektor b keduanya sudah diketahui dari `enc_flag.txt`. Tinggal pindah ruas:

$$Ax = y - b \quad \text{sehingga} \quad x = A^{-1}(y - b)$$

Solve pakai Gauss-Jordan dengan aritmetika Fraction (bukan float), supaya hasilnya integer bersih tanpa galat pembulatan.



```

1  # ----- Part 3:  $y = A.x + b$  -----
2  def solve_part3():
3      P = DATA["part3_linear_algebra"]
4      A, b, y = P["A"], P["b"], P["y"]
5      n = len(y)
6      M = [[Fraction(v) for v in A[i]] + [Fraction(y[i] - b[i])] for i in range(n)]
7      M = rref(M, 1)
8      return "".join(chr(int(M[i][n])) for i in range(n))
9

```

Screenshot: fungsi solve_part3() dan fungsi rref() di solve.py

Hasil: mber_4t_SMK_Immanuel

4. PART 2 - MATRIX

Plaintext di-reshape menjadi matriks P berukuran 4x5, lalu dikalikan key K berukuran 4x4. Key K ikut dibocorkan di dalam enc_flag.txt, jadi cukup solve $K \times P = C$ dengan metode eksak yang sama seperti Part 3.



```

1  # ----- Part 2:  $C = K . P$  -----
2  def solve_part2():
3      C, K = DATA["part2_matrix"]
4      M = [[Fraction(v) for v in K[i]] + [Fraction(v) for v in C[i]] for i in range(4)]
5      M = rref(M, 5)
6      return "".join(chr(int(M[i][4 + j])) for i in range(4) for j in range(5))
7

```

Screenshot: fungsi solve_part2() di solve.py

Hasil: m3_t0_ctf_f0r_n3w_m3

5. PART 1 - CALCULUS (BAGIAN PALING MENARIK)



```
1  # Definite Integral
2  def calculus(data_ord):
3      n = len(data_ord)
4      cipher_integrals = []
5
6      for k in range(1, n + 1):
7          val = sum(Fraction(a, i + 1) * (k ** (i + 1)) for i, a in enumerate(data_ord))
8          cipher_integrals.append(float(val))
9
10     return cipher_integrals
```

Screenshot: fungsi `calculus()` di `enc.py` (baris 17-25)

Baris intinya adalah:

```
val = sum(Fraction(a, i + 1) * (k ** (i + 1)) for i, a in enumerate(data_ord))
```

Ini tidak lain adalah integral tentu dari polinomial $p(x) = a_0 + a_1.x + a_2.x^2 + \dots + a_{19}.x^{19}$, dihitung dari 0 sampai k :

$V(k) = \text{integral dari 0 sampai } k \text{ dari } p(x) \, dx$

$$= (a_0.k^1)/1 + (a_1.k^2)/2 + (a_2.k^3)/3 + \dots + (a_{19}.k^{20})/20$$

dengan k berjalan dari 1 sampai 20. Jadi ada 20 persamaan dan 20 unknown, membentuk matriks M dengan $M[k][i] = k^{(i+1)} / (i+1)$ yang secara teori invertible.

Masalahnya: output disimpan sebagai float lewat float(val). Matriks ini bertipe Vandermonde dengan condition number $4,7 \times 10^{26}$, sedangkan float64 hanya punya sekitar 16 digit signifikan. Artinya galat pembulatan sekecil apa pun saat enkripsi akan diperbesar sampai jauh melampaui nilai koefisien yang dicari. Solve langsung memakai np.linalg.solve atau np.linalg.lstsq menghasilkan angka sampah berorde 10^{16} , sama sekali di luar rentang ASCII. Hanya 2 dari 20 koefisien yang kebetulan jatuh di rentang 32 sampai 126.

```
Ukuran sistem      : 20 persamaan, 20 unknown
Condition number M  : 4.679e+26
Presisi float64     : ~16 digit signifikan

Hasil np.linalg.solve (seharusnya integer di rentang 32-126):
[ 5.837e+14 -4.094e+15  9.379e+15 -1.122e+16  8.385e+15 -4.286e+15  1.583e+15 -4.380e+14  9.284e+13
 -1.531e+13  1.983e+12 -2.025e+11  1.628e+10 -1.025e+09  4.990e+07 -1.839e+06  4.965e+04 -8.140e+02
 1.096e+02  4.794e+01]

Dipaksa jadi karakter:
'????????????????n0'

Hanya 2/20 koefisien yang jatuh di rentang ASCII printable.
=> solve langsung GAGAL, perlu pendekatan lain (DFS + interval pruning).
```

Screenshot: output dari menjalankan python naive_fail.py

Solusinya adalah memanfaatkan satu constraint yang belum dipakai sama sekali: setiap koefisien $a[i]$ pasti berupa karakter ASCII printable, yaitu berada di rentang 32 sampai 126. Constraint ini mengubah masalah numerik yang ill-conditioned menjadi search problem yang bisa di-prune habis.

Pencarian dilakukan secara DFS dari koefisien indeks tertinggi (a_{19}) turun ke terendah (a_0). Urutan ini dipilih karena pada $k = 20$, bobot $20^{(i+1)/(i+1)}$ membuat suku berindeks tinggi sangat mendominasi, sehingga rentang kandidatnya paling sempit.

Di setiap level DFS, untuk setiap persamaan k dihitung:

$R(k) = V(k)$ dikurangi kontribusi koefisien yang sudah dipilih

S = jumlah $M[k][i]$ untuk semua i yang belum dipilih

Sisa kontribusi pasti berada di rentang $[32 \times S, 126 \times S]$

Sehingga kandidat untuk koefisien $a[j]$ terbatas pada:

$$a[j] \geq (R(k) - 126 \times S - \text{tol}) / M[k][j]$$

$$a[j] \leq (R(k) - 32 \times S + \text{tol}) / M[k][j]$$

Rentang dari kedua puluh persamaan diinterseksikan, dan biasanya hanya menyisakan 0 sampai 2 kandidat per level. Seluruh aritmetika memakai Fraction agar eksak, dengan toleransi $\text{tol} = |V(k)| / 2^{50}$ untuk menampung galat pembulatan float64 yang terjadi saat enkripsi.

```
1 def dfs(j, R):
2     if j == 0:
3         if all(abs(R[k]) <= TOL[k] for k in range(n)):
4             sols.append(list(cur))
5         return
6     lo, hi = LO, HI
7     for k in range(n):
8         c = M[k][j - 1]
9         a_lo = (R[k] - HI * S[k][j - 1] - TOL[k]) / c
10        a_hi = (R[k] - LO * S[k][j - 1] + TOL[k]) / c
11        lo = max(lo, -(-a_lo.numerator // a_lo.denominator)) # ceil
12        hi = min(hi, a_hi.numerator // a_hi.denominator) # floor
13        if lo > hi:
14            return
15        for a in range(lo, hi + 1):
16            cur[j - 1] = a
17            dfs(j - 1, [R[k] - a * M[k][j - 1] for k in range(n)])
```

Screenshot: fungsi `solve_part1()` di `solve.py`, terutama bagian fungsi `dfs()`

Hasil pencarian: hanya 32.802 node yang dikunjungi dan solusinya unik, selesai dalam hitungan detik.

Hasil: C2SI{e4sy_m4th_w3lc0

6. GABUNG DAN VERIFIKASI

Keempat part digabung berurutan:

```
part1: C2SI{e4sy_m4th_w3lc0
part2: m3_t0_ctf_f0r_n3w_m3
part3: mber_4t_SMK_Immanuel
part4: _Pontianak_G00DLUCK}
```

```
part1: C2SI{e4sy_m4th_w3lc0
part2: m3_t0_ctf_f0r_n3w_m3
part3: mber_4t_SMK_Immanuel
part4: _Pontianak_G00DLUCK}

FLAG: C2SI{e4sy_m4th_w3lc0m3_t0_ctf_f0r_n3w_m3mber_4t_SMK_Immanuel_Pontianak_G00DLUCK}
```

Screenshot: hasil menjalankan python solve.py

Untuk memastikan tidak ada satu karakter pun yang meleset, flag hasil recovery dienkrpsi ulang memakai enc.py asli, lalu output-nya dibandingkan dengan enc_flag.txt dari panitia. Proses re-encrypt dijalankan di folder temporary supaya file asli tidak tertimpa, karena enc.py menulis ke enc_flag.txt di direktori kerjanya.

Satu catatan teknis: di Windows, mode tulis teks Python mengubah setiap \n menjadi \r\n, sedangkan file dari panitia memakai LF. Jadi perbandingan byte dilakukan setelah line ending dinormalkan terlebih dahulu, ditambah perbandingan struktur JSON hasil parsing. Keduanya cocok, dengan sha256 yang sama persis.

```
Flag hasil recovery : C2SI{e4sy_m4th_w3lc0m3_t0_ctf_f0r_n3w_m3mber_4t_SMK_Immanuel_Pontianak_G00DLUCK}
Panjang              : 80 karakter

Re-encrypt dengan enc.py asli di folder temp ...
  output enc.py: done!

enc_flag.txt asli      byte  sha256 (setelah normalisasi line ending)
enc_flag.txt baru      11197  88ad85d3e6a443015ac8c395ae6ea29ea705e489a523c279dca84c9ef7724f56
enc_flag.txt baru      11197  88ad85d3e6a443015ac8c395ae6ea29ea705e489a523c279dca84c9ef7724f56

Identik byte per byte : True
Identik struktur JSON : True

>>> MATCH - flag terkonfirmasi benar.
```

Screenshot: output dari menjalankan python verify.py, sampai baris MATCH

7. FLAG

C2SI{e4sy_m4th_w3lc0m3_t0_ctf_f0r_n3w_m3mber_4t_SMK_Immanuel_Pontianak_G00DLUCK}

8. TAKEAWAY

Pertama, encoding matematis yang linear bukanlah enkripsi. Ketika key ikut dikirim bersama ciphertext seperti pada Part 2 dan Part 3, yang terjadi hanyalah encoding reversible biasa.

Kedua, penyimpanan hasil sebagai float memang membocorkan lebih sedikit informasi daripada yang diperkirakan pembuat soal, tetapi struktur plaintext, dalam hal ini fakta bahwa setiap byte pasti ASCII printable, selalu menjadi constraint tambahan yang cukup untuk menutup kekurangan presisi tersebut.